



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине «Машинное обучение»
на тему «Библиотеки Python для машинного обучения. Работа с данными»

Выполнил:

Студент Солодкий М.И.

Группа ИПБП41

Проверил:

Дольников Максим Геннадьевич

Ростов-на-Дону

2026 г.

Цель работы:

Изучить основные библиотеки Python для машинного обучения (NumPy, Pandas, Matplotlib, SciPy, scikit-learn, TensorFlow, Keras, PyTorch) и приобрести практические навыки работы с табличными данными: чтение/запись, предобработка, статистический анализ, тензорные вычисления, обработка изображений.

Задачи:

1. Освоить чтение и запись данных в различных форматах.
2. Научиться выявлять и обрабатывать проблемы в данных.
3. Реализовать собственные функции масштабирования и сравнить их со sklearn.
4. Освоить статистический анализ данных средствами NumPy, Pandas, SciPy.
5. Научиться работать с тензорами TensorFlow, PyTorch, Keras.
6. Освоить базовую обработку изображений средствами SciPy.

Выполнение работы:

Раздел 1

Вариант №14

Задание 1. Чтение файлов.

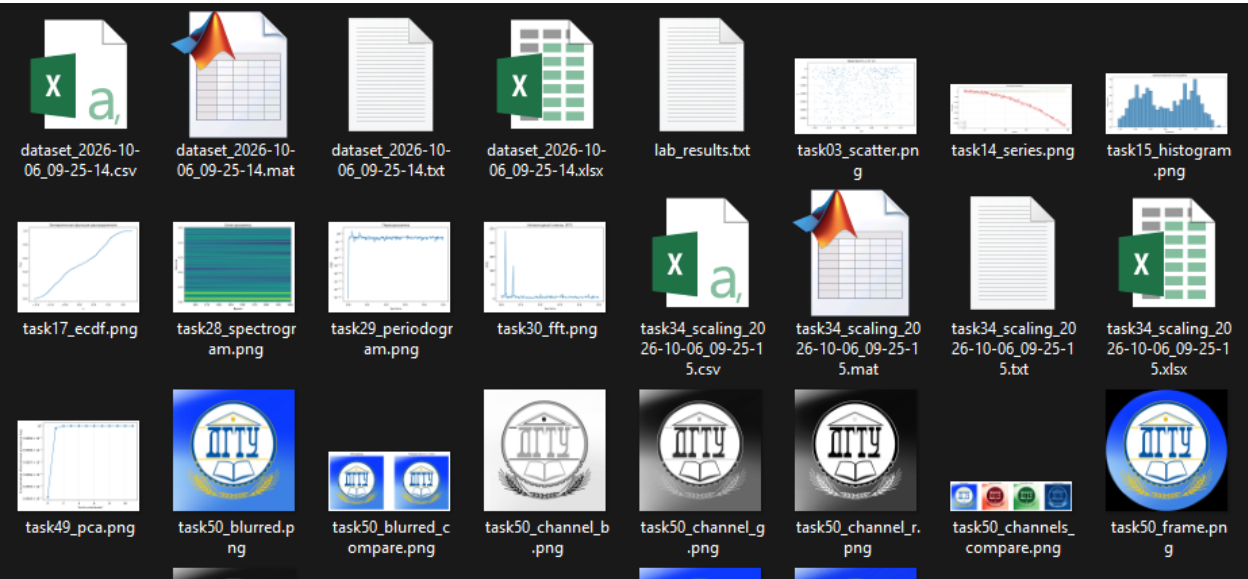
Реализована универсальная функция чтения табличных данных, которая по расширению файла выбирает подходящий способ загрузки: для csv используется `pandas.read_csv`, для xlsx — `pandas.read_excel`, для txt — `pandas.read_csv` с разделителем из пробелов, для mat — `scipy.io.loadmat` с автоматическим выбором первого двумерного массива. Датасет `dataset.csv`

содержит 500 строк и 7 столбцов: id, timestamp, category, x1, x2, x4, y. Данные загружены успешно, структура корректная.

1	id,timestamp,category,x1,x2,x4,y						
2	1,	2024-01-01 00:00:00,	A,	0.6835119515543189,	0.17517845559778916,-	0.8096035199216461,	38.30243095992137
3	2,	2024-01-01 01:00:00,	A,	3.7523416784317725,	0.2022088913599509,	0.4250610444640897,	181.42270213903245
4	3,	2024-01-01 02:00:00,	B,	1.9008476763721005,	0.7691980363788655,-	0.4698393470709161,	132.61413327478337
5	4,	2024-01-01 03:00:00,	A,	-1.1538073113248062,	1.1543328354359643,-	0.005452270878279103,-	16.476769933677367
6	5,	2024-01-01 04:00:00,	C,	-1.796829342696716,	1.2278867700250227,	0.5622842480036996,-	76.58910720230492
7	6,	2024-01-01 05:00:00,	B,	0.9838383430130114,	1.2159268897025404,	0.03142226981093782,	63.58725856668329
8	7,	2024-01-01 06:00:00,	A,	-2.6404664140412843,	1.3894581524202798,-	0.4003858905265686,-	61.28433639916948
9	8,	2024-01-01 07:00:00,	C,	3.6629175317087075,	1.3609807106267675,-	0.06060980392583083,	199.6449210190394
10	9,	2024-01-01 08:00:00,	A,	2.358880241442574,	1.3197207580220327,-	0.02436295920533532,	142.26211751007563
11	10,	2024-01-01 09:00:00,	A,	-0.9383513042094096,	0.9727677591046489,-	0.2890110298821277,-	3.7302428912153047
12	11,	2024-01-01 10:00:00,	A,	-3.426269058181755,	0.6047560323050803,-	0.15879606266710236,-	128.45897746791246
13	12,	2024-01-01 11:00:00,	A,	2.7077447483308257,	0.6024408263963378,	1.719647170504717,	119.12886533076065
14	13,	2024-01-01 12:00:00,	A,	-0.22907969050523577,	0.8681917902100307,	0.7049194479412387,	0.004412668841205392
15	14,	2024-01-01 13:00:00,	A,	2.4756326239469235,	0.2625353374085602,-	0.12648031802916035,	116.52441997340075

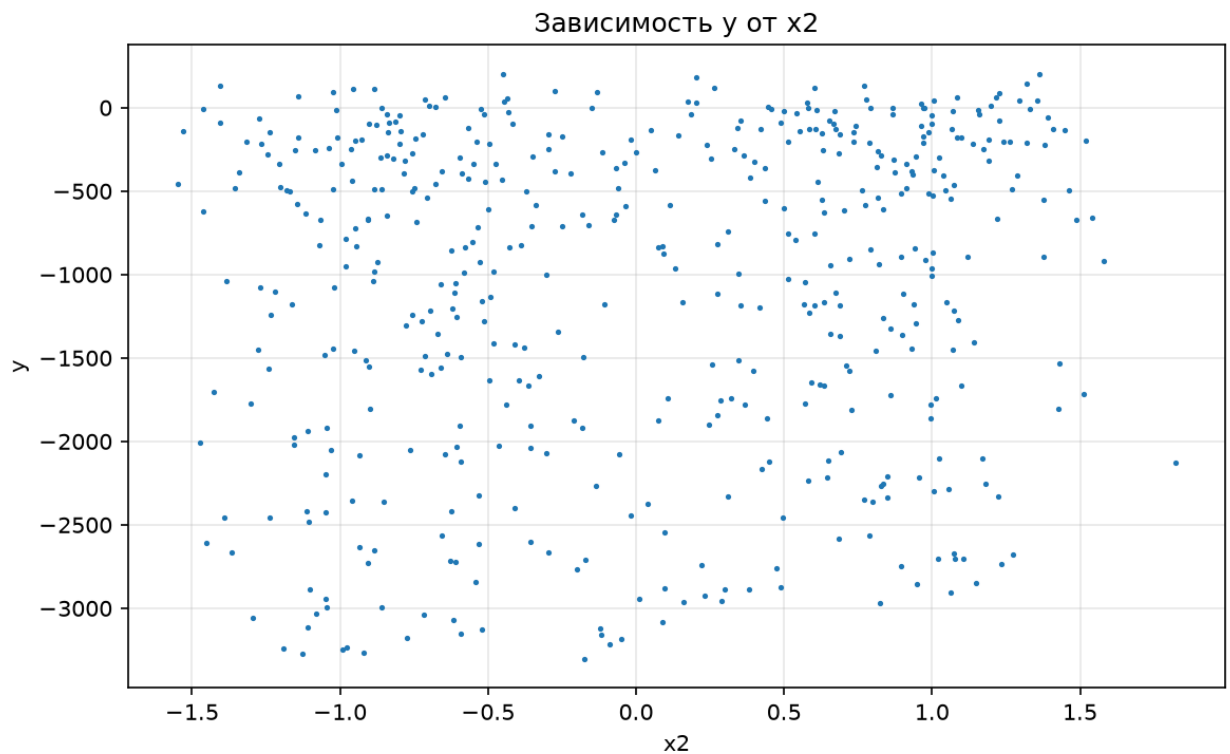
Задание 2. Запись файлов.

Реализована функция, сохраняющая таблицу сразу во все поддерживаемые форматы (xlsx, csv, txt, mat). К имени каждого файла добавляется метка времени, чтобы результаты разных запусков не перезаписывали друг друга.



Задание 3. Сохранение изображений (графиков).

Реализована функция сохранения графиков Matplotlib в PNG с высоким разрешением. В качестве примера построена и сохранена диаграмма рассеяния зависимости y от x_2 .



Задание 4. Проверка данных.

Реализована функция проверки качества данных: она подсчитывает пропуски по каждому столбцу, выводит типы данных и находит некорректные числовые значения. В датасете обнаружены пропуски в столбцах x_1 и x_2 , а также строка «ошибка» в столбце x_1 , из-за которой столбец имеет тип object.

```
=====
ЗАДАНИЕ 4. ПРОВЕРКА ДАННЫХ
=====

Пропущенные значения:
{'id': 0, 'timestamp': 0, 'category': 0, 'x1': 0, 'x2': 10, 'x4': 0, 'y': 0}

Типы данных:
{'id': 'int64', 'timestamp': 'str', 'category': 'str', 'x1': 'str', 'x2': 'float64', 'x4': 'float64', 'y': 'float64'}

Некорректные числовые значения:
{}
```

Задание 5. Вывод таблицы.

Реализована функция вывода первых n строк таблицы, её размера и типов столбцов.

```
=====
ЗАДАНИЕ 5. ВЫВОД ТАБЛИЦЫ
=====

   id  timestamp category      x1      x2      x4      y
0   1  2024-01-01 00:00:00      A  0.6835119515543189  0.175178 -0.809604  38.302431
1   2  2024-01-01 01:00:00      A  3.7523416784317725  0.202209  0.425061 181.422702
2   3  2024-01-01 02:00:00      B  1.9008476763721005  0.769198 -0.469839 132.614133
3   4  2024-01-01 03:00:00      A -1.1538073113248062  1.154333 -0.005452 -16.476770
4   5  2024-01-01 04:00:00      C -1.796829342696716  1.227887  0.562284 -76.589107

Shape: (500, 7)
Dtypes:
id          int64
timestamp   str
category    str
x1          str
x2          float64
x4          float64
y          float64
dtype: object
```

Задание 6. Извлечение столбцов по варианту.

Реализована функция извлечения подмножества столбцов. В датасете нет столбца x3, поэтому для анализа выбраны реально существующие: x1, x2, x4, y.

```
=====
ЗАДАНИЕ 6. ИЗВЛЕЧЕНИЕ СТОЛБЦОВ
=====

      x1      x2      y
0  0.6835119515543189  0.175178  38.302431
1  3.7523416784317725  0.202209 181.422702
2  1.9008476763721005  0.769198 132.614133
3 -1.1538073113248062  1.154333 -16.476770
4 -1.796829342696716  1.227887 -76.589107
```

Задание 7. Явные типы данных.

Реализована функция приведения столбцов к заданным типам. Числовые признаки приведены к float64.

```

=====
ЗАДАНИЕ 7. ЯВНЫЕ ТИПЫ ДАННЫХ
=====
x2    float64
y      float64
dtype: object
      x2      y
0  0.175178  38.302431
1  0.202209  181.422702
2  0.769198  132.614133
3  1.154333  -16.476770
4  1.227887  -76.589107

```

Раздел 2

Задание 8. Обработка проблемных данных.

Реализована функция, поддерживающая стратегии drop, cast, mean, ffill, bfill, zero, value. В столбце x1 строка «ошибка» сначала превращена в NaN (стратегия cast), затем пропуски заполнены средним значением (стратегия mean).

```

=====
ЗАДАНИЕ 8. ОБРАБОТКА ПРОБЛЕМНЫХ ДАННЫХ
=====
x1 после очистки:
0    0.683512
1    3.752342
2    1.900848
3   -1.153807
4   -1.796829
Name: x1, dtype: float64

Осталось NaN в x1: 0

```

Задание 9. DataFrame → NumPy.

Реализована функция преобразования DataFrame в массив NumPy. Полученный массив имеет форму (n, 4) и тип float64.

```

=====
ЗАДАНИЕ 9. DATAFRAME -> NUMPY
=====
Тип: <class 'numpy.ndarray'>
Shape: (490, 4)
[[ 6.83511952e-01  1.75178456e-01 -8.09603520e-01  3.83024310e+01]
 [ 3.75234168e+00  2.02208891e-01  4.25061044e-01  1.81422702e+02]
 [ 1.90084768e+00  7.69198036e-01 -4.69839347e-01  1.32614133e+02]
 [-1.15380731e+00  1.15433284e+00 -5.45227088e-03 -1.64767699e+01]
 [-1.79682934e+00  1.22788677e+00  5.62284248e-01 -7.65891072e+01]]

```

Задание 10. Функции масштабирования.

Реализованы собственные классы `MinMaxScalerCustom` и `StandardScalerCustom` с методами `fit`, `transform`, `fit_transform`, `inverse_transform`. Проведено масштабирование исходного массива обоими способами.

```
=====
ЗАДАНИЕ 10. МАСШТАБИРОВАНИЕ
=====
MinMax (первые 5 строк):
[[0.52610197 0.51134446 0.00144849 0.95381688]
 [0.79176621 0.51936559 0.00639876 0.99467738]
 [0.631485   0.68761641 0.00281074 0.98074265]
 [0.36704784 0.80190298 0.00467266 0.93817755]
 [0.31138234 0.82372969 0.00694895 0.92101562]]

StandardScaler (первые 5 строк):
[[ 0.33547939  0.17766022 -1.12022748  1.14909669]
 [ 1.86829323  0.20978368 -1.10359078  1.29536088]
 [ 0.94351216  0.88360382 -1.11564927  1.24548014]
 [-0.5822217   1.34130498 -1.10939181  1.09311416]
 [-0.90339725  1.4287178  -1.10174174  1.03168133]]
```

Задание 11. Восстановление масштаба.

Проверено, что `inverse_transform` возвращает исходные значения с точностью до $1e-10$.

```
=====
ЗАДАНИЕ 11. ВОССТАНОВЛЕНИЕ МАСШТАБА
=====
[[ 6.83511952e-01  1.75178456e-01 -8.09603520e-01  3.83024310e+01]
 [ 3.75234168e+00  2.02208891e-01  4.25061044e-01  1.81422702e+02]
 [ 1.90084768e+00  7.69198036e-01 -4.69839347e-01  1.32614133e+02]
 [-1.15380731e+00  1.15433284e+00 -5.45227088e-03 -1.64767699e+01]
 [-1.79682934e+00  1.22788677e+00  5.62284248e-01 -7.65891072e+01]]

Совпадает с исходными: True
```

Задание 12. Разбиение на 3 части.

Массив разбит на `train` / `validation` / `test` в пропорции 70 / 15 / 15 без перемешивания, чтобы сохранить порядок.

```
=====
ЗАДАНИЕ 12. РАЗБИЕНИЕ НА 3 ЧАСТИ
=====
Train: (343, 4)
Validation: (74, 4)
Test: (73, 4)
```

Задание 13. Фиксация seed.

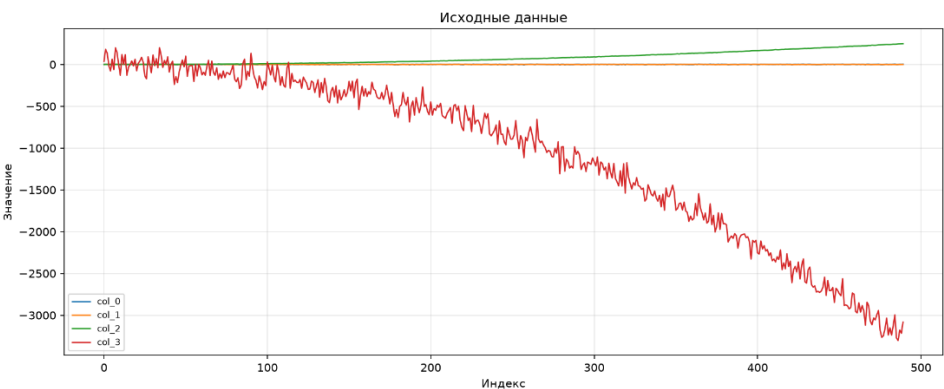
Реализована функция, фиксирующая состояние всех генераторов случайных чисел (random, numpy, TensorFlow, PyTorch). Проверено, что два последовательных вызова дают одинаковые массивы случайных чисел.

```
=====
ЗАДАНИЕ 13. ФИКСАЦИЯ SEED
=====
Первый массив: [0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
Второй массив: [0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
Одинаковые: True
```

Раздел 3

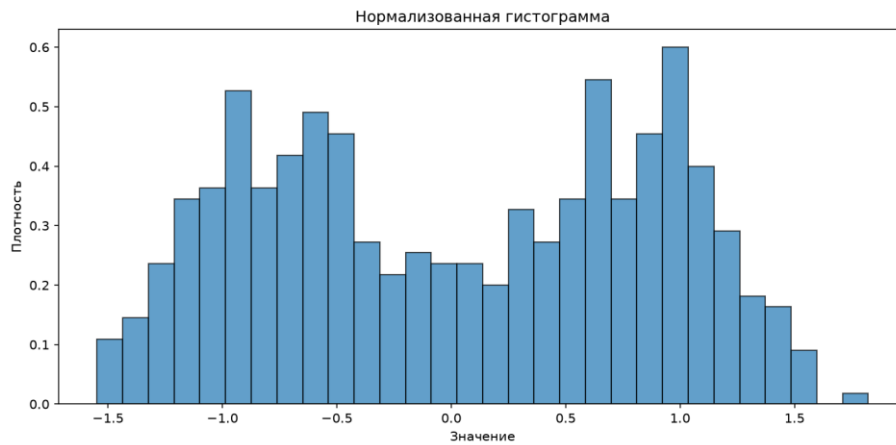
Задание 14. График исходных данных.

Построен линейный график всех четырёх столбцов массива X.



Задание 15. Гистограмма и плотность.

Построена нормализованная гистограмма столбца x2.



Задание 16. Отсортированные столбцы.

Выведены отсортированные значения x_2 (первые 15).

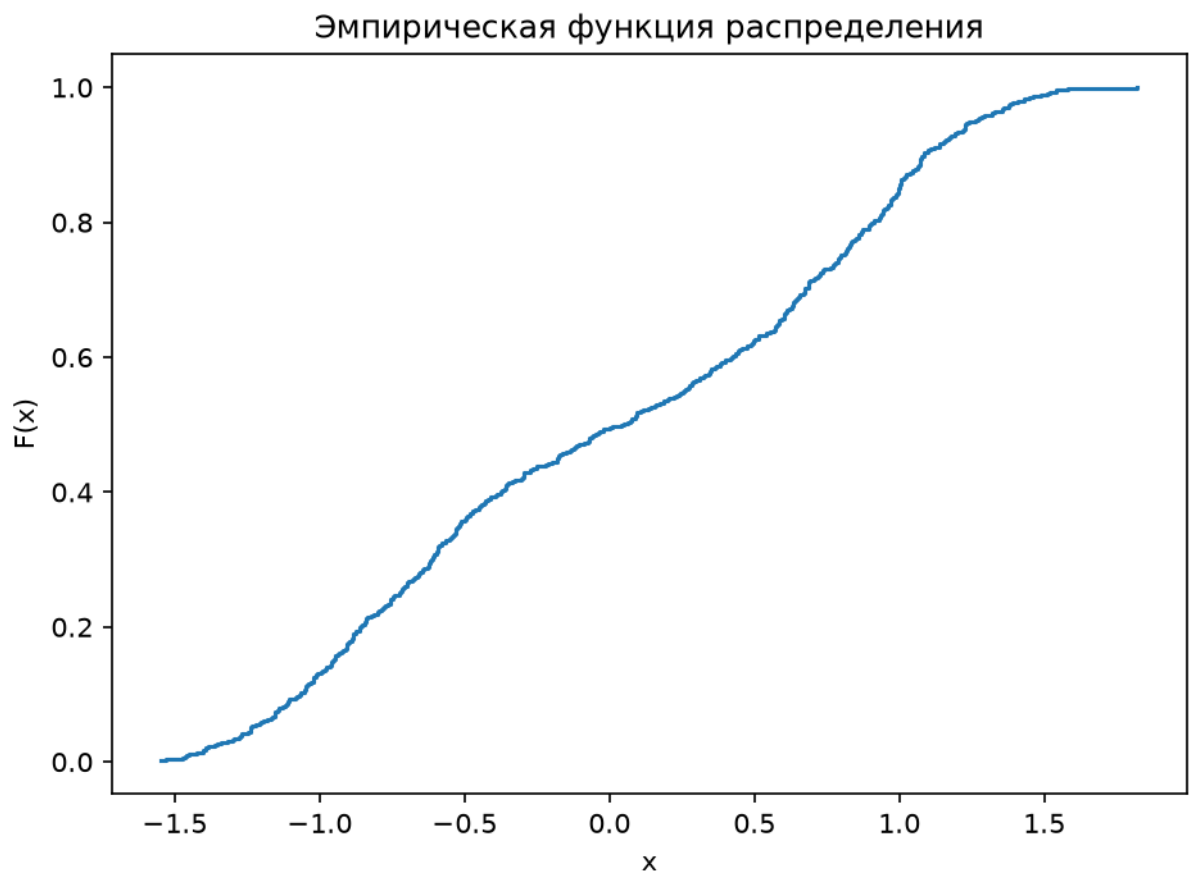
```
=====
ЗАДАНИЕ 16. СОРТИРОВКА
```

```
=====
Отсортированные значения  $x_2$  (первые 15):
```

```
[-1.54800342 -1.53121426 -1.4747505  -1.46243712 -1.46130653 -1.45001478
-1.42559295 -1.40382521 -1.40359561 -1.3926516  -1.38370154 -1.3642829
-1.35534391 -1.34061818 -1.31603314]
```

Задание 17. Эмпирическая функция распределения.

Построена ECDF для x_2 .



Задание 18. Статистики.

Вычислены среднее, дисперсия, мода и медиана для x_2 .

```
=====
ЗАДАНИЕ 18. СТАТИСТИКИ
=====
mean: 0.02568542082404832
var: 0.7080460602948297
mode: -1.548003418133002
median: 0.0448501088557536
```

Задание 19. Доверительные интервалы.

Построены доверительные интервалы для среднего (через распределение Стьюдента) и для дисперсии (через распределение хи-квадрат) при уровне значимости 0,05.

```
=====
ЗАДАНИЕ 19. ДОВЕРИТЕЛЬНЫЕ ИНТЕРВАЛЫ
=====
Интервал для среднего: (np.float64(-0.04907997914399774), np.float64(0.10045082079209437))
Интервал для дисперсии: (np.float64(0.6283305253627959), np.float64(0.8075534793793682))
```

Задание 20. Ковариация и корреляция.

Вычислены ковариационная и корреляционная матрицы массива X.

```
=====
ЗАДАНИЕ 20. КОВАРИАЦИЯ И КОРРЕЛЯЦИЯ
=====
Ковариационная матрица:
[[ 4.01655741e+00 -4.82086756e-02  1.21616039e+01  1.97390521e+01]
 [-4.82086756e-02  7.09494007e-01 -5.85703202e+00  9.32880527e+01]
 [ 1.21616039e+01 -5.85703202e+00  5.51887892e+03 -7.24348619e+04]
 [ 1.97390521e+01  9.32880527e+01 -7.24348619e+04  9.59430435e+05]]

Корреляционная матрица:
[[ 1.          -0.02855774  0.08168427  0.01005524]
 [-0.02855774  1.          -0.09360042  0.11306936]
 [ 0.08168427 -0.09360042  1.          -0.99544085]
 [ 0.01005524  0.11306936 -0.99544085  1.          ]]
```

Задание 21. Значимость коэффициента корреляции.

Проверена значимость корреляции между x_2 и y с помощью критерия Пирсона. Получены коэффициент корреляции и p-value.

```
=====
ЗАДАНИЕ 21. ЗНАЧИМОСТЬ КОРРЕЛЯЦИИ
=====
r = 0.113069
p-value = 0.012261
Корреляция значима.
```

Задание 22. Взаимная корреляция.

Рассчитана взаимная корреляция между x_2 и y для лагов от -50 до 50.

```
=====
ЗАДАНИЕ 22. ВЗАИМНАЯ КОРРЕЛЯЦИЯ
=====
Лаги (первые 10):      [-50 -49 -48 -47 -46 -45 -44 -43 -42 -41]
Корреляции (первые 10): [0.0932 0.097  0.0954 0.0919 0.0866 0.0811 0.0738 0.0617 0.0544 0.0453]
```

Задание 23. Производная и градиент.

Вычислена производная x_2 и многомерный градиент массива X.

```
=====
ЗАДАНИЕ 23. ПРОИЗВОДНАЯ И ГРАДИЕНТ
=====
```

```
Производная x2 (первые 10): [ 0.02703044  0.29700979  0.47606197  0.22934437  0.03079703  0.08078569
 0.07252691 -0.0348687 -0.19410648 -0.35748236]
Градиент X (первые 3 строки):
[[ 3.06882973e+00  2.70304358e-02  1.23466456e+00  1.43120271e+02]
 [ 6.08667862e-01  2.97009790e-01  1.69882086e-01  4.71558512e+01]
 [-2.45307449e+00  4.76061972e-01 -2.15256658e-01 -9.89497360e+01]]
```

Задание 24. Свёртка.

Выполнена свёртка x_2 и y в режиме full.

```
=====
ЗАДАНИЕ 24. СВЁРТКА
=====
```

```
Свёртка (первые 20 значений): [ 6.7097607  39.52644087  89.37857721 207.69312173 341.71125042
405.39856018 360.84185289 428.30872938 459.19810627 530.16653588
564.51160793 551.03086276 445.44295566 573.06141048 559.08826189
596.3226262 521.36522175 476.00393511 432.33315539 675.5359009 ]
```

Задание 25. Скалярное и векторное произведения.

Вычислены скалярное и векторное произведения для первых трёх компонент x_2 и y .

```
=====
ЗАДАНИЕ 25. ПРОИЗВЕДЕНИЯ
=====
```

```
Скалярное произведение: 31948.426673881273
Векторное произведение: [-112.73422937  6.23101563  24.03625667]
```

Задание 26. Нормы L1 и L2.

Вычислены L1- и L2-нормы для x_2 .

```
=====
ЗАДАНИЕ 26. НОРМЫ L1 И L2
=====
```

```
L1: 369.12428672379417
L2: 18.635070232158817
```

Задание 27. Проверка гипотез о распределении.

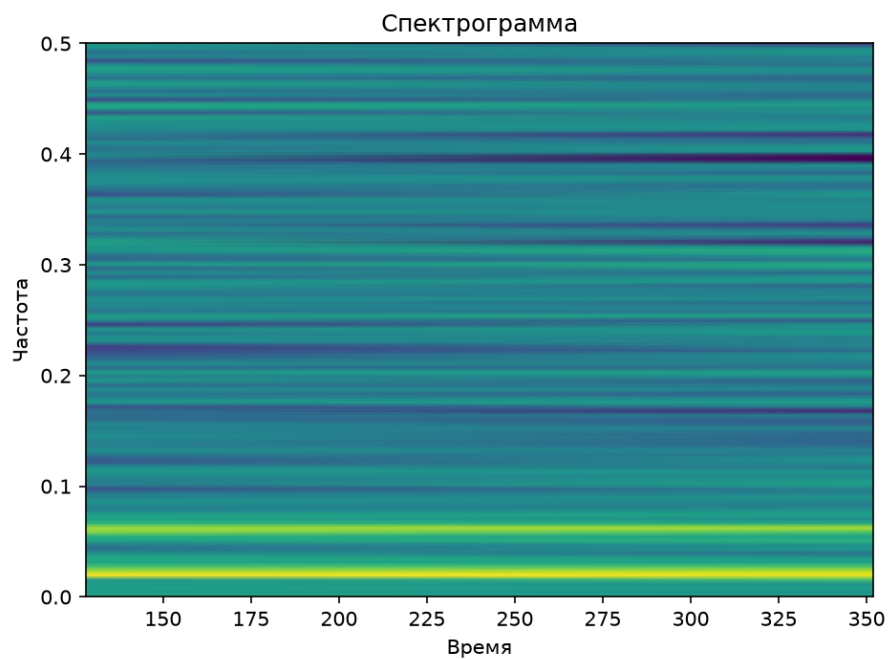
Проверены гипотезы: для x_1 — о равномерном распределении (критерий Колмогорова–Смирнова), для x_2 — о нормальном распределении (Колмогоров–Смирнов и Шапиро–Уилк).

```
=====
ЗАДАНИЕ 27. ПРОВЕРКА ГИПОТЕЗ
=====
```

```
ks_stat_uniform: 0.206798
p_uniform: 0.000000
ks_stat_normal: 0.101447
p_normal: 0.000076
shapiro_stat: 0.945357
p_shapiro: 0.000000
```

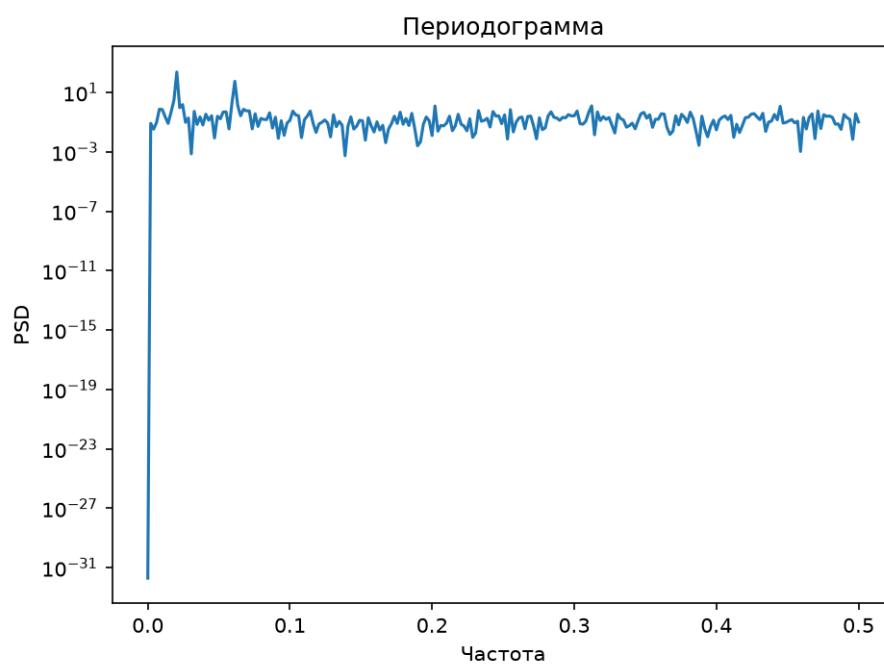
Задание 28. Спектрограмма.

Построена спектрограмма для x_2 .



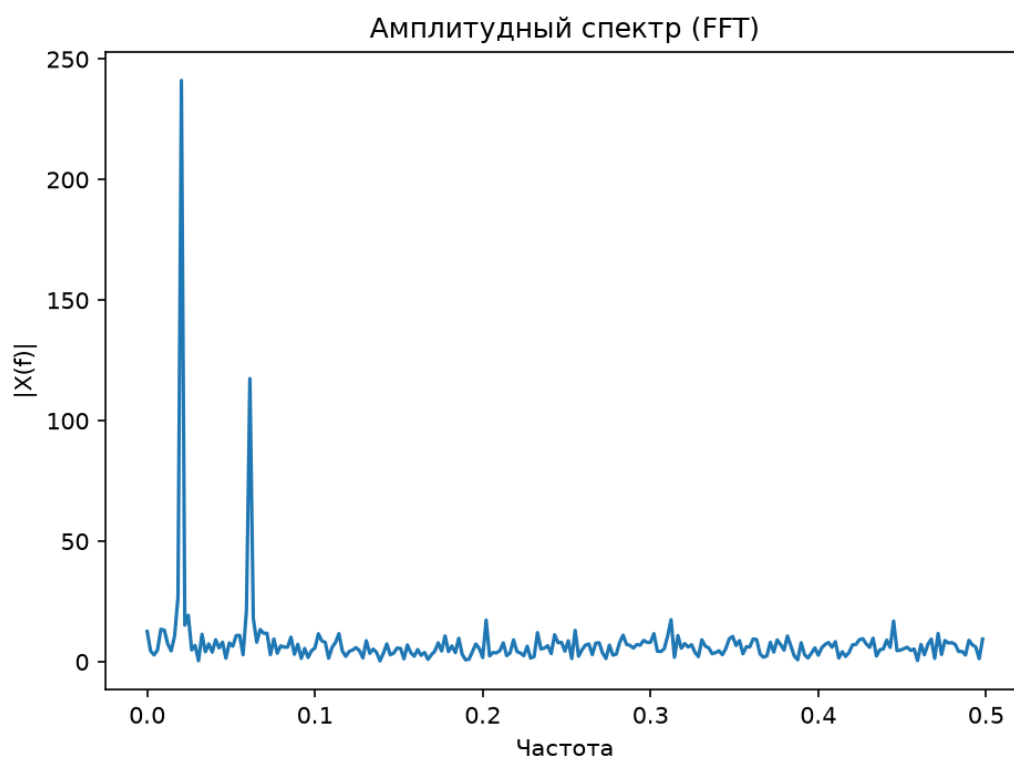
Задание 29. Периодограмма.

Построена периодограмма для x_2 .



Задание 30. АЧХ через FFT.

Построен амплитудный спектр через быстрое преобразование Фурье.



Задание 31. Кубическая интерполяция.

Выполнена кубическая интерполяция x^2 с увеличением числа точек в 10 раз.

```
=====
ЗАДАНИЕ 31. КУБИЧЕСКАЯ ИНТЕРПОЛЯЦИЯ
=====
```

```
Исходных точек: 490
Новых точек:    4900
```

Задание 32. Интерполяция сплайнами.

Выполнена сплайн-интерполяция x^2 ($k = 3$, $s = 0$).

```
=====
ЗАДАНИЕ 32. ИНТЕРПОЛЯЦИЯ СПЛАЙНАМИ
=====
```

```
Количество точек: 4900
```

Задание 33. Бинарные маски.

Построены маски: положительные, отрицательные, нулевые, попадающие в интервал $[-1; 1]$.

```
=====
ЗАДАНИЕ 33. БИНАРНЫЕ МАСКИ
=====
```

```
positive: 248
negative: 242
zero: 0
in_[-1, 1]: 354
```

Задание 34. Сравнение собственного и sklearn-масштабирования.

Проведено сравнение собственных классов MinMaxScalerCustom и StandardScalerCustom с аналогами из sklearn. Результаты совпадают с точностью до $1e-10$. Таблица сравнения сохранена в файл.

```
=====
ЗАДАНИЕ 34. СРАВНЕНИЕ МАСШТАБИРОВАНИЯ
=====
```

```
MinMax custom == sklearn: True
Standard custom == sklearn: True
```

Задание 35. Инверсия.

Проверено, что обратное преобразование восстанавливает исходные данные и для собственного класса, и для sklearn.

```
=====
ЗАДАНИЕ 35. ИНВЕРСИЯ
=====
```

```
Custom восстановление: True
Sklearn восстановление: True
```

Задание 36. Скользящее окно.

Построена матрица окон ширины 5 для x2.

```
=====
ЗАДАНИЕ 36. СКОЛЬЗЯЩЕЕ ОКНО
=====
```

```
Shape: (486, 5)
[[0.17517846 0.20220889 0.76919804 1.15433284 1.22788677]
 [0.20220889 0.76919804 1.15433284 1.22788677 1.21592689]
 [0.76919804 1.15433284 1.22788677 1.21592689 1.38945815]
 [1.15433284 1.22788677 1.21592689 1.38945815 1.36098071]
 [1.22788677 1.21592689 1.38945815 1.36098071 1.31972076]]
```

Задание 37. Скользящее среднее.

Вычислено скользящее среднее ширины 5.

```
=====
ЗАДАНИЕ 37. СКОЛЬЗЯЩЕЕ СРЕДНЕЕ
=====
```

```
Shape: (486,)
[0.705761 0.91391068 1.15136054 1.26971707 1.30279466 1.25177085
 1.12953668 0.97213322 0.87357543 0.66213835]
```

Задание 38. Тензор TensorFlow.

Создан тензор X_tf из массива X.

Вставить: скриншот вывода тензора и его формы.

```

=====
ЗАДАНИЕ 38. ТЕНЗОР TENSORFLOW
=====
I0000 00:00:1791269478.479007    8500 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
tf.Tensor(
[[ 6.8351197e-01  1.7517845e-01 -8.0960351e-01  3.8302429e+01]
 [ 3.7523417e+00  2.0220889e-01  4.2506105e-01  1.8142270e+02]
 [ 1.9008477e+00  7.6919806e-01 -4.6983933e-01  1.3261414e+02]
 ...,
 [ 2.0883217e+00 -7.7371609e-01  2.4696034e+02 -3.1775527e+03]
 [ 1.3637830e+00 -8.7367907e-02  2.4803680e+02 -3.2159512e+03]
 [ 3.6934147e+00  8.9787774e-02  2.4824251e+02 -3.0819236e+03]], shape=(490, 4), dtype=float32)
Shape: (490, 4)

```

Задание 39. Тензор PyTorch.

Создан тензор X_pt из массива X.

```

=====
ЗАДАНИЕ 39. ТЕНЗОР PYTORCH
=====
tensor([[ 6.8351e-01,  1.7518e-01, -8.0960e-01,  3.8302e+01],
        [ 3.7523e+00,  2.0221e-01,  4.2506e-01,  1.8142e+02],
        [ 1.9008e+00,  7.6920e-01, -4.6984e-01,  1.3261e+02],
        ...,
        [ 2.0883e+00, -7.7372e-01,  2.4696e+02, -3.1776e+03],
        [ 1.3638e+00, -8.7368e-02,  2.4804e+02, -3.2160e+03],
        [ 3.6934e+00,  8.9788e-02,  2.4824e+02, -3.0819e+03]])
Shape: torch.Size([490, 4])

```

Задание 40. Случайные тензоры.

Созданы случайные тензоры A ($k \times n$), W ($m \times p$) и B ($k \times p$).

```

=====
ЗАДАНИЕ 40. СЛУЧАЙНЫЕ ТЕНЗОРЫ
=====
A: (5, 490) W: (4, 4) B: (5, 4)

```

Задание 41. Операция AXW+B (TensorFlow).

Вычислено произведение A @ X @ W + B средствами TensorFlow.

```

=====
ЗАДАНИЕ 41. AXW+B — TENSORFLOW
=====
tf.Tensor(
[[ 9.8803638e+05  4.5342275e+06  1.0982832e+06 -1.4741044e+06]
 [ 1.0485796e+06  4.8122260e+06  1.1652555e+06 -1.5648160e+06]
 [ 1.0355451e+06  4.7523165e+06  1.1515402e+06 -1.5445019e+06]
 [ 1.0051203e+06  4.6122970e+06  1.1176095e+06 -1.4991235e+06]
 [ 9.4782794e+05  4.3496845e+06  1.0538730e+06 -1.4137750e+06]], shape=(5, 4), dtype=float32)
Shape: (5, 4)

```

Задание 42. Операция AXW+B (PyTorch).

Та же операция выполнена в PyTorch.

```
=====
ЗАДАНИЕ 42. AX+B – PYTORCH
=====
tensor([[ 1012548.0000,   982663.7500,   1577941.5000,  -4667055.0000],
        [ 1011822.5000,   982528.5625,   1576521.3750,  -4664496.5000],
        [  961577.4375,   934481.6875,   1498967.3750,  -4434239.5000],
        [  994291.2500,   966740.6875,   1550000.6250,  -4585725.5000],
        [  954685.0625,   927434.4375,   1487612.1250,  -4401842.0000]])
Shape: torch.Size([5, 4])
```

Задание 43. Матрицы T, P, Q.

Созданы три случайные матрицы формы 3×10 .

```
=====
ЗАДАНИЕ 43. МАТРИЦЫ T, P, Q
=====
T:
[[0.15599452 0.05808361 0.86617615 0.60111501 0.70807258 0.02058449
  0.96990985 0.83244264 0.21233911 0.18182497]
 [0.18340451 0.30424224 0.52475643 0.43194502 0.29122914 0.61185289
  0.13949386 0.29214465 0.36636184 0.45606998]
 [0.78517596 0.19967378 0.51423444 0.59241457 0.04645041 0.60754485
  0.17052412 0.06505159 0.94888554 0.96563203]]

P:
[[0.80839735 0.30461377 0.09767211 0.68423303 0.44015249 0.12203823
  0.49517691 0.03438852 0.9093204 0.25877998]
 [0.66252228 0.31171108 0.52006802 0.54671028 0.18485446 0.96958463
  0.77513282 0.93949894 0.89482735 0.59789998]
 [0.92187424 0.0884925 0.19598286 0.04522729 0.32533033 0.38867729
  0.27134903 0.82873751 0.35675333 0.28093451]]

Q:
[[0.54269608 0.14092422 0.80219698 0.07455064 0.98688694 0.77224477
  0.19871568 0.00552212 0.81546143 0.70685734]
 [0.72900717 0.77127035 0.07404465 0.35846573 0.11586906 0.86310343
  0.62329813 0.33089802 0.06355835 0.31098232]
 [0.32518332 0.72960618 0.63755747 0.88721274 0.47221493 0.11959425
  0.71324479 0.76078505 0.5612772 0.77096718]]
```

Задание 44. Операция в Keras.

Вычислено выражение $V = |\sin(T) - \exp(P) \cdot \sqrt{Q}|$ средствами Keras.

```
=====
ЗАДАНИЕ 44. ОПЕРАЦИЯ KERAS
=====
tf.Tensor(
[[1.49797235 0.45102772 0.22569209 0.02432931 0.89235761 0.97225411
  0.09341165 0.66266681 2.03114421 0.90823724]
 [1.47375672 0.89986746 0.04327207 0.61568717 0.12238065 1.87533608
  1.57485154 1.18385263 0.25866487 0.57356411]
 [0.72665377 0.73485325 0.47947753 0.42713246 0.90495725 0.06075312
  0.9381096 1.93277175 0.25758052 0.3404482 ]], shape=(3, 10), dtype=float64)
Shape: (3, 10)
```

Задание 45. Та же операция в PyTorch.

Результат повторён в PyTorch.

```
=====
ЗАДАНИЕ 45. ОПЕРАЦИЯ PYTORCH
=====
tensor([[1.4980, 0.4510, 0.2257, 0.0243, 0.8924, 0.9723, 0.0934, 0.6627, 2.0311,
         0.9082],
        [1.4738, 0.8999, 0.0433, 0.6157, 0.1224, 1.8753, 1.5749, 1.1839, 0.2587,
         0.5736],
        [0.7267, 0.7349, 0.4795, 0.4271, 0.9050, 0.0608, 0.9381, 1.9328, 0.2576,
         0.3404]], dtype=torch.float64)
Shape: torch.Size([3, 10])
```

Задание 46. Разреженный массив SciPy.

Создана разреженная матрица размером $100\,000 \times 100\,000$ с плотностью $1e-5$. Выведены её размер, число ненулевых элементов, тип, формат и плотность.

```
=====
ЗАДАНИЕ 46. РАЗРЕЖЕННАЯ МАТРИЦА
=====
Shape:      (100000, 100000)
NNZ:        100000
Dtype:      float64
Формат:     csr
Плотность:  1e-05
```

Задание 47. Обратная матрица.

Создана случайная матрица 100×100 , найдены её определитель и обратная матрица. Проверено условие $M @ M^{-1} = I$.

```
=====
ЗАДАНИЕ 47. ОБРАТНАЯ МАТРИЦА
=====
Определитель: -2.894539561757214e+24
Обратима:     True
Проверка M @ M^-1 = I: True
```

Задание 48. Достроение массива F.

Собран трёхмерный массив F формы (n, m, 3): [исходные данные, Min-Max, StandardScaler]. Для PCA выполнено преобразование к форме (n, m·3).

```
=====
ЗАДАНИЕ 48. МАССИВ F
=====
```

```
Shape F: (490, 4, 3)
Shape X_F (для PCA): (490, 12)
```

Задание 49. Пайплайн sklearn + PCA.

Для двух пайплайнов (MinMax + PCA и Standard + PCA) вычислены значения логарифмического правдоподобия. Построен график кумулятивной объяснённой дисперсии в логарифмическом масштабе.

```
=====
ЗАДАНИЕ 49. ПАЙПЛАЙН SKLEARN + PCA
=====
```

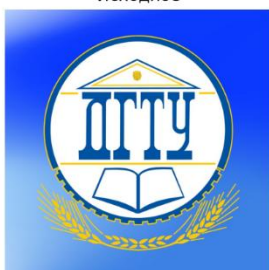
```
Log-likelihood по вариантам:
  minmax: 51529.4906
  standard: 48771.9894
Лучший вариант: minmax
График сохранён: C:\Users\AKSI\Desktop\Progs\Lab1_1\output\task49_pca.png
```

Задание 50. Обработка изображения средствами SciPy.

Для цветного изображения image.jpg выполнены десять операций: разделение на каналы R, G, B; поворот на 40° с бикубической интерполяцией; перевод в оттенки серого; построение гистограммы яркостей чёрно-белого изображения (256 бинов); разбиение изображения на три части по квантилям гистограммы; наложение чёрной круговой рамки; добавление гауссовского шума ($\sigma = 25$); гауссовское размытие ($\sigma = 2$); операция sharpening через unsharp masking (amount = 1,5); PCA к чёрно-белому изображению (32 компоненты), построение кривой MSE и сетки из 16 первых главных компонент. Все обработанные изображения сохранены отдельными PNG-файлами, а также в виде сравнительных фигур «до и после».



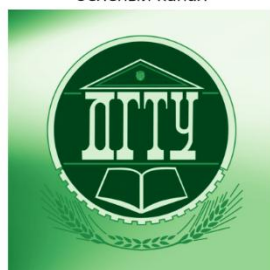
Исходное



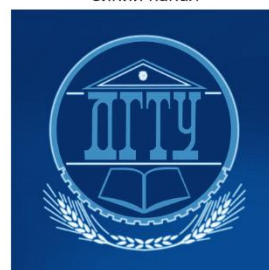
Красный канал



Зелёный канал



Синий канал



Исходное



Поворот на 40.0°



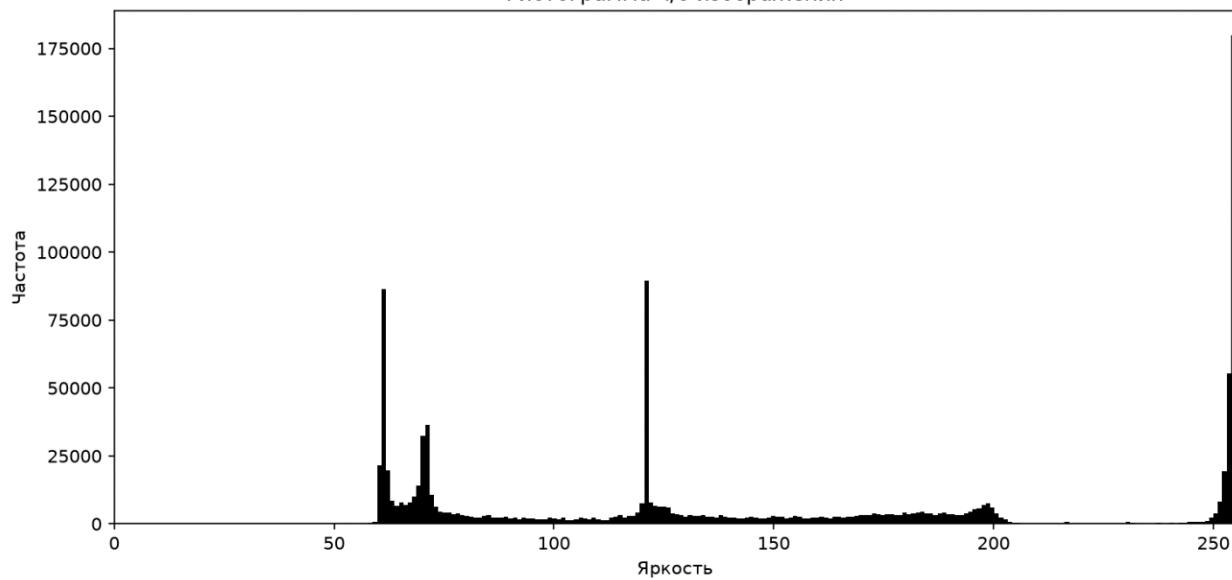
Исходное



Ч/б



Гистограмма ч/б изображения



Тёмные (≤ 98.61328125)



Средние ($98.61328125 - 185.2734375$)



Светлые (> 185.2734375)



Исходное



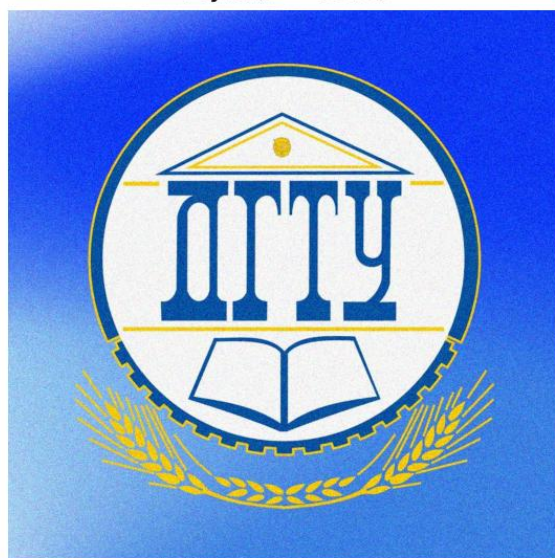
В круговой рамке



Исходное



Шум ($\sigma = 25.0$)



Исходное



Размытие ($\sigma = 2.0$)



Исходное



Sharpening (amount=1.5)

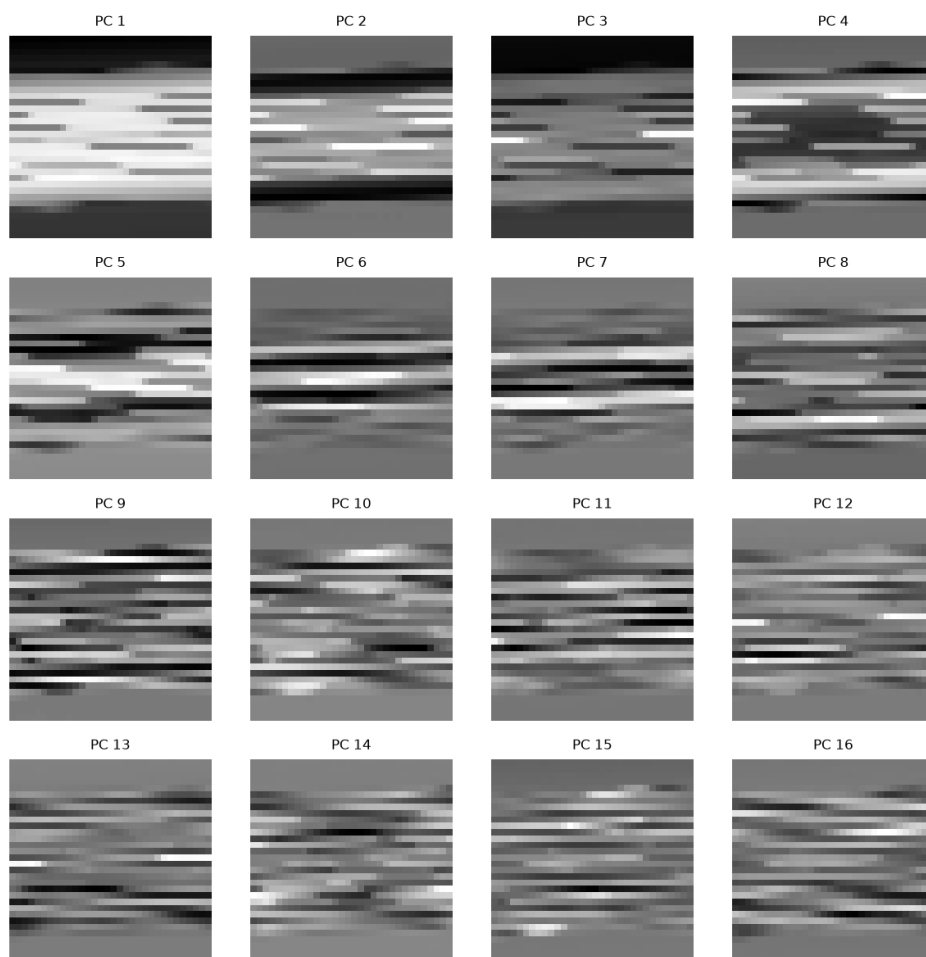
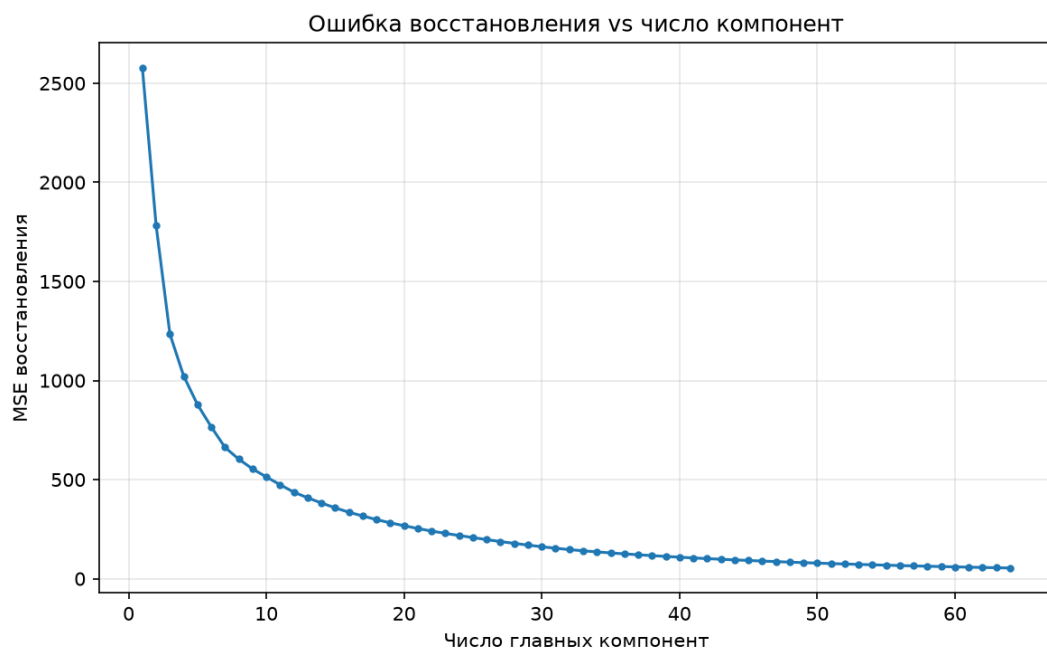


Исходное Ч/Б



РСА (компонент: 32)





Отправление выполненного задания на GitHub.

Был создан удаленный репозиторий, а после запущены все файлы выполненной лабораторной.

```
AKSI@HOME-PC MINGW64 ~
$ cd Desktop/

AKSI@HOME-PC MINGW64 ~/Desktop
$ cd Progs/

AKSI@HOME-PC MINGW64 ~/Desktop/Progs
$ cd Lab1_1

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1
$ git init
Initialized empty Git repository in C:/Users/AKSI/Desktop/Progs/Lab1_1/.git/

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ nano README.md

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git add README.md

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git commit -m "Первый коммит, добавлен README"
[main (root-commit) 39bb221] Первый коммит, добавлен README
1 file changed, 97 insertions(+)
create mode 100644 README.md

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git config user.name
7AKSI7

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git config user.email
solodkijmaksim64@gmail.com

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git remote add origin https://github.com/7AKSI7/MachineLearningLab1.git

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 2.71 KiB | 2.71 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/7AKSI7/MachineLearningLab1.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    data/
    main.py
    output/
    requirements.txt
    src/

nothing added to commit but untracked files present (use "git add" to track)
```

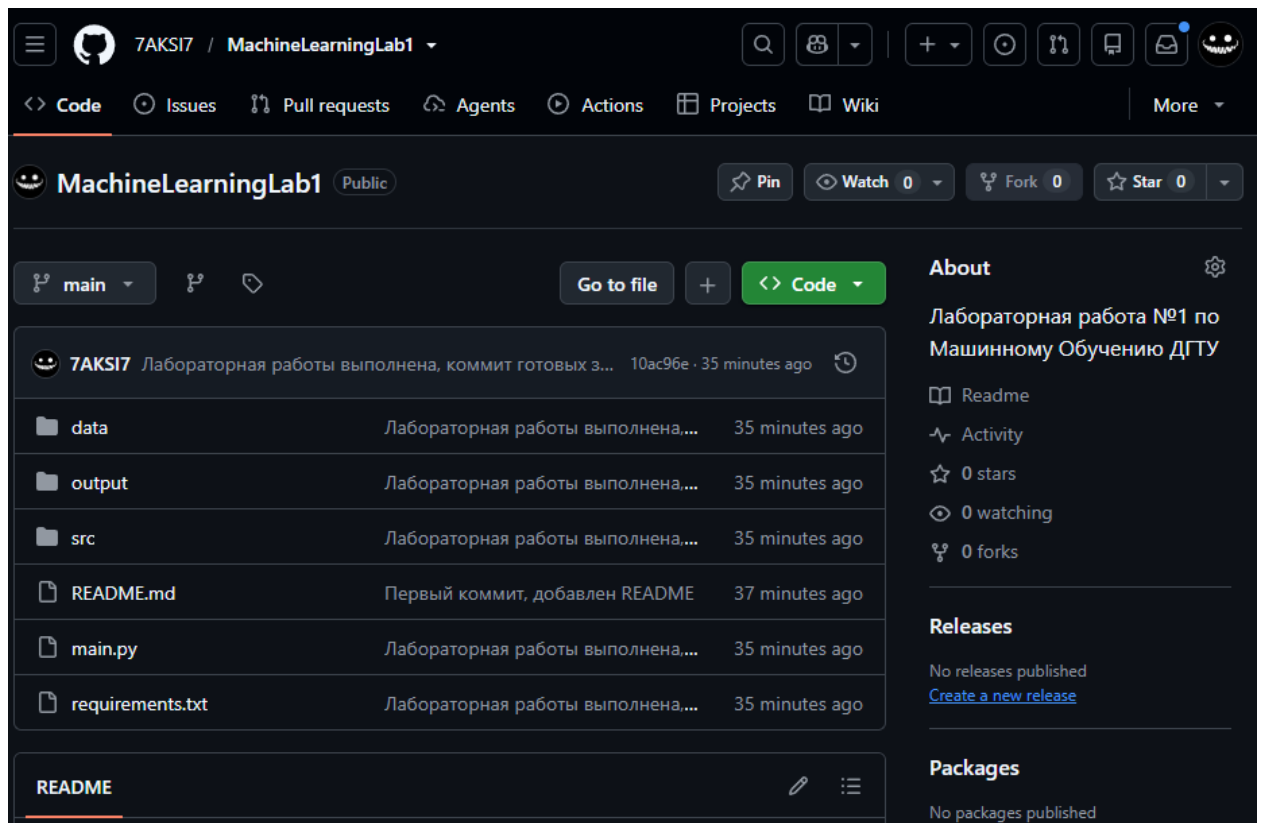
```
AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git add -A
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'requirements.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/config.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/data_loader.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/image_proc.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/io_utils.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/matrix_ops.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/preprocess.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/scaling.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/stats_analysis.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/tensors.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/windows.py', LF will be replaced by CRLF the next time Git touches it

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   data/dataset.csv
    new file:   data/image.jpg
    new file:   main.py
    new file:   output/dataset_2026-10-06_09-25-14.csv
    new file:   output/dataset_2026-10-06_09-25-14.mat
    new file:   output/dataset_2026-10-06_09-25-14.txt
    new file:   output/dataset_2026-10-06_09-25-14.xlsx
    new file:   output/lab_results.txt
    new file:   output/task03_scatter.png
    new file:   output/task14_series.png
    new file:   output/task15_histogram.png
    new file:   output/task17_ecdf.png
    new file:   output/task28_spectrogram.png
    new file:   output/task29_periodogram.png
    new file:   output/task30_fft.png
    new file:   output/task34_scaling_2026-10-06_09-25-15.csv
```

```
AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git commit -m "Лабораторная работы выполнена, коммит готовых заданий"
[main 10ac96e] Лабораторная работы выполнена, коммит готовых заданий
77 files changed, 4645 insertions(+)
create mode 100644 data/dataset.csv
create mode 100644 data/image.jpg
create mode 100644 main.py
create mode 100644 output/dataset_2026-10-06_09-25-14.csv
create mode 100644 output/dataset_2026-10-06_09-25-14.mat
create mode 100644 output/dataset_2026-10-06_09-25-14.txt
create mode 100644 output/dataset_2026-10-06_09-25-14.xlsx
create mode 100644 output/lab_results.txt
create mode 100644 output/task03_scatter.png
create mode 100644 output/task14_series.png
create mode 100644 output/task15_histogram.png
create mode 100644 output/task17_ecdf.png
create mode 100644 output/task28_spectrogram.png
```

```
AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git push origin
Enumerating objects: 83, done.
Counting objects: 100% (83/83), done.
Delta compression using up to 12 threads
Compressing objects: 100% (81/81), done.
Writing objects: 100% (82/82), 12.18 MiB | 2.52 MiB/s, done.
Total 82 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/7AKSI7/MachineLearningLab1.git
   39bb221..10ac96e  main -> main
```



Вывод

В ходе лабораторной работы были изучены и практически применены основные библиотеки Python для машинного обучения. Освоены чтение и запись табличных данных в четырёх форматах; предобработка данных, включая обработку пропусков, приведение типов, масштабирование и разбиение на выборки; статистический анализ, включая описательные статистики, доверительные интервалы, ковариацию и корреляцию, проверку гипотез о распределении, спектральный анализ и интерполяцию; тензорные вычисления в TensorFlow, PyTorch и Keras; работа с разреженными матрицами, обратной матрицей и PCA-пайплайном; базовая обработка изображений средствами SciPy. Все задания выполнены, результаты сохранены в папку output, код воспроизводим благодаря фиксации seed.